

PickReady, Cloud Run Production Deployment Runbook

Manual, copy-paste deployment using **only the gcloud CLI and your existing gcloud login**. No service-account JSON key is downloaded or used anywhere.

Project	<code>pick-ready-503913</code> (number <code>1034326377358</code>)
Region	<code>asia-south1</code>
Deploying as	<code>manjuchro@gmail.com</code> (already authenticated)
Shell	Windows PowerShell 5.1
Total time	~2 hours, most of it waiting on Cloud SQL

Every command is a **single line**. There are no `\` continuations, no `&&`, and no Bash-isms. Paste one line at a time.

`infra/gcp/deploy.sh` is the scripted alternative to this document. It does the same work but **requires a downloaded service-account key**, which this runbook deliberately avoids. The two are otherwise equivalent, and this runbook fixes three defects found in the script while writing it (see [Appendix A](#)).

Read this before you start

Three things were verified against the running code and this GCP project while writing this document. All three contradict a plausible assumption.

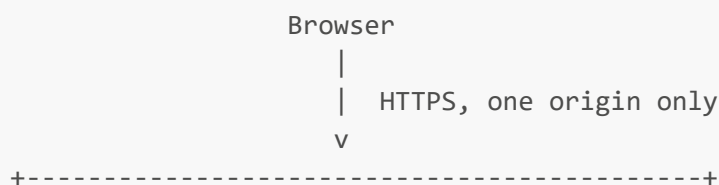
1. The health endpoint is `/health`, not `/api/v1/health`. `backend/app/main.py:147` registers it on the app root, outside both router prefixes. The live OpenAPI document lists 189 paths and `/health` is the only one matching. `curl $BACKEND_URL/api/v1/health` returns **404** and is not a useful check.

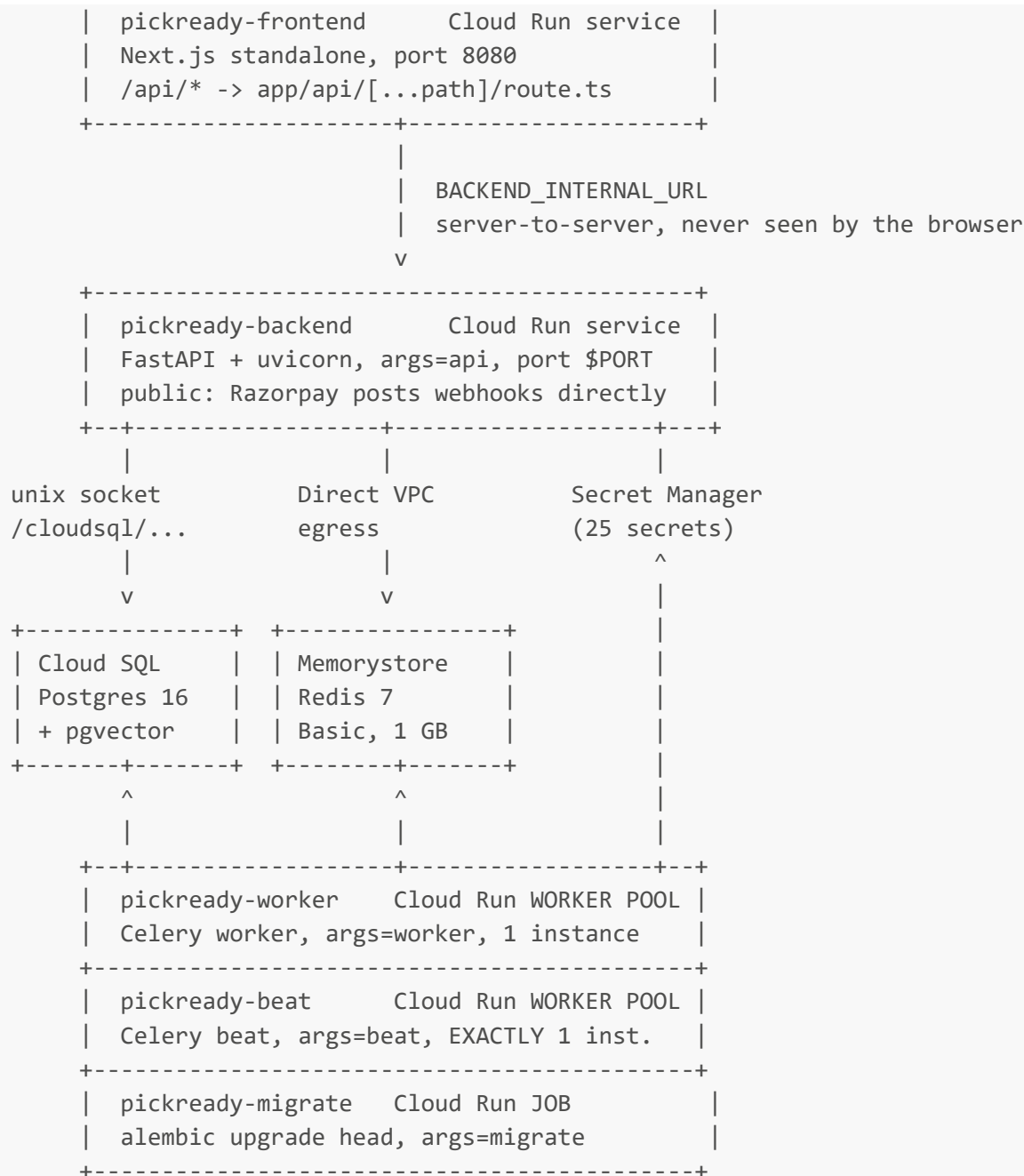
2. The Razorpay webhook path is `/api/v1/billing/webhook/razorpay`.

`backend/app/api/billing.py:581` declares `@router.post("/webhook/razorpay")` and the router mounts at `/api/v1/billing`. The path without the trailing `/razorpay` does not exist.

3. `gcloud run worker-pools deploy` is currently broken on this machine. It fails with `No module named 'grpc'`. See [Phase 0](#). Celery cannot be deployed until this is fixed, and it fails at Phase 12 of 15 if you skip it.

Architecture Overview





Two images, five workloads. backend/Dockerfile serves four of them; the role is chosen by the container args (backend/docker-entrypoint.sh).

Why the shapes are what they are

These four are not stylistic. Changing any of them breaks the deployment.

Celery runs in worker pools, not services. A Cloud Run *service* must listen on `$PORT` and pass a startup probe. A Celery worker serves no HTTP at all, so the revision never goes ready and rolls back forever. Beat is pinned to exactly one instance: two schedulers means every scheduled task fires twice, so every reminder email goes out twice and every credit reconciliation runs twice.

The browser never talks to the backend directly. It calls `/api/*` on the frontend origin and `frontend/app/api/[...path]/route.ts` forwards to `BACKEND_INTERNAL_URL`. That is what lets `COOKIE_SAMESITE` stay `strict`: `*.a.run.app` is on the Public Suffix List, so two Cloud Run services are **cross-**

site, and a split origin makes the browser silently drop every auth cookie. Users would appear to log in and be instantly logged out.

Cloud SQL is reached over a unix socket. `--add-cloudsql-instances` mounts a socket at `/cloudsql/<CONNECTION_NAME>`. There is no host and port to connect to, and the app uses asyncpg, so the DSN must name the driver:

```
postgresql+asyncpg://USER:PASS@/pickready?host=/cloudsql/PROJECT:REGION:INSTANCE
```

A `postgresql://user:pass@host:5432/db` DSN fails 100% of the time here.

Memorystore has a private VPC address only. Reaching it needs Direct VPC egress (`--network / --subnet`), which replaces the older Serverless VPC Access connector and costs nothing extra.

Pre-Deployment Checklist

Run all six. Every one must match the expected output before you continue.

```
gcloud auth list
```

Expect **ACTIVE:** * beside `manjuchro@gmail.com`.

```
gcloud config list
```

Expect `project = pick-ready-503913` and `region = asia-south1`.

```
docker --version
```

Expect **Docker version 29.x** or newer. Docker must be **running**, not just installed, because Phases 6 builds images locally.

```
gcloud projects describe pick-ready-503913 --format="value(projectNumber)"
```

Expect `1034326377358`. A permission error here means your account cannot deploy and nothing after this point will work.

```
Test-Path C:\dev\pickready\.env ; Test-Path C:\dev\pickready\frontend\.env.local
```

Expect **True** twice.

```
Select-String -Path C:\dev\pickready\.env -Pattern "^POSTGRES_PASSWORD=." -Quiet
```

Expect **True**. Phase 2 cannot create the database user without it.

Variables **.env** must contain

25 of these are pushed to Secret Manager in Phase 5. **POSTGRES_PASSWORD** is **not** pushed: it is used once, locally, to create the SQL user, and then only ever reaches the cloud inside the **DATABASE_URL** secret.

Group	Names
Deploy-only	POSTGRES_PASSWORD
Auth	JWT_SECRET, LLM_KEY_ENCRYPTION_SECRET, FIREBASE_SERVICE_ACCOUNT_JSON
Email	SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD, SMTP_FROM_EMAIL, SMTP_FROM_NAME
Payments	RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET
Storage / search / SMS	CLOUDINARY_URL, TAVILY_API_KEY, MSG91_API_KEY, MSG91_SENDER_ID
Embeddings	BGE_M3_ENDPOINT
LLM roster	GROQ_API_KEY_1..7, GEMINI_API_KEY_1..7, OPENROUTER_API_KEY_1..7

Every LLM slot is optional; the router enumerates only populated ones. Blank entries are skipped, not pushed as empty secrets.

Firebase config **frontend/.env.local** must contain

These six are **build-time** arguments, not runtime environment. They are inlined into the JS bundle by the compiler, and **frontend/Dockerfile:50** fails the build outright if the API key or project id is missing. They are public by design; access is controlled by Firebase Auth rules and the authorised-domain list, not by hiding them.

```
NEXT_PUBLIC_FIREBASE_API_KEY
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN
NEXT_PUBLIC_FIREBASE_PROJECT_ID
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID
NEXT_PUBLIC_FIREBASE_APP_ID
```

Two blockers found in the current **.env**

Both are real and both were confirmed by reading the code. Fix them now, not after the deploy.

BLOCKER 1: RAZORPAY_WEBHOOK_SECRET is empty. `backend/app/api/billing.py:594` calls `verify_webhook_signature`, which returns `False` immediately when the secret is blank (`services/razorpay.py:236`). Line 595 then reads `if settings.is_production or ...` and raises **400 Invalid webhook signature**. In production a blank secret does not mean "skip verification", it means **every Razorpay webhook is rejected**. Subscriptions are charged by Razorpay and never credited in PickReady.

Fix: create the webhook in the Razorpay dashboard first (Phase 15 step 2), copy the signing secret it gives you into `.env`, and push it in Phase 5.

BLOCKER 2: FIREBASE_SERVICE_ACCOUNT_JSON is wrapped in single quotes. The value in `.env` begins `'{"type":"service_account"...` and ends `'`. Docker Compose's `env_file` parser strips those quotes, which is why local login works. `backend/app/services/firebase_auth.py:31` calls `json.loads(raw)` with **no** stripping, so if the quotes reach Secret Manager, Firebase Admin raises `Firebase Admin could not be initialized` and **nobody can sign in to production**.

Fix: the Phase 5 script below strips matched surrounding quotes. Do not replace it with a naive `Select-String/sed` read.

Phase 0: Fix the local gcloud install (2 minutes)

`gcloud run worker-pools deploy` currently fails on this machine:

```
ERROR: gcloud failed to load (gcloud.run.worker-pools.deploy):
Problem loading gcloud.run.worker-pools.deploy: No module named 'grpc'.
```

The cause is not a missing package. `grpc` is installed (1.81.0) in the Python gcloud uses (`pythoncore-3.14-64`), but gcloud runs with `CLOUDSDK_PYTHON_SITEPACKAGES` unset, so it never puts site-packages on `sys.path`. Note that `gcloud run worker-pools list` works fine, because only `deploy` imports the gRPC client, which is why this stays hidden until Phase 12.

Set it for the session:

```
$env:CLOUDSDK_PYTHON_SITEPACKAGES = "1"
```

Verify:

```
gcloud run worker-pools deploy --help
```

Success: the help text prints, beginning `NAME gcloud run worker-pools deploy`. If it still errors, use the SDK's own bundled interpreter instead:

```
$env:CLOUDSDK_PYTHON = "C:\Program Files (x86)\Google\Cloud SDK\google-cloud-sdk\platform\bundledpython\python.exe"
```

To make it permanent across terminals:

```
[Environment]::SetEnvironmentVariable("CLOUDSDK_PYTHON_SITEPACKAGES", "1", "User")
```

Phase 1: Enable Required APIs (5 minutes, likely already done)

All nine were verified enabled on `pick-ready-503913`. Run this anyway: it is idempotent, and it is cheap insurance against a project that has drifted.

```
gcloud services enable run.googleapis.com artifactregistry.googleapis.com  
sqladmin.googleapis.com redis.googleapis.com secretmanager.googleapis.com  
cloudbuild.googleapis.com compute.googleapis.com vpcaccess.googleapis.com  
servicenetworking.googleapis.com
```

Wait: 0 seconds if already enabled, up to 3 minutes otherwise. **Success:** `Operation "operations/..." finished successfully.`, or no output.

Verify:

```
gcloud services list --enabled --format="value(config.name)" | Select-String  
"run.googleapis|sqladmin|redis.googleapis|secretmanager|vpcaccess"
```

Success: five lines. Fewer means an API failed to enable; re-run the command above before continuing.

Phase 2: Create Cloud SQL, Postgres 16 (10 minutes)

Set the session variables first. Every later phase reuses them, so keep this terminal open. If you close it, re-run this block.

```
$PROJECT_ID = "pick-ready-503913"
```

```
$REGION = "asia-south1"
```

```
$SQL_INSTANCE = "pickready-postgres"
```

```
$SQL_DB = "pickready"
```

```
$SQL_USER = "pickready"
```

Create the instance. **This is the slowest step in the entire deployment.**

```
gcloud sql instances create $SQL_INSTANCE --database-version=POSTGRES_16 --  
tier=db-custom-1-3840 --region=$REGION --storage-auto-increase --enable-point-in-  
time-recovery --backup-start-time=02:00 --project=$PROJECT_ID
```

Wait: 7 to 12 minutes. The command blocks and prints `Creating Cloud SQL instance for POSTGRES_16...working...`. Do not interrupt it.

Use `--enable-point-in-time-recovery`, never `--enable-bin-log`. The latter is MySQL-only and is rejected outright on a Postgres instance.

Verify it is `RUNNABLE`:

```
gcloud sql instances describe $SQL_INSTANCE --format="value(state)" --  
project=$PROJECT_ID
```

Success: `RUNNABLE`. Anything else (`PENDING_CREATE`, `FAILED`) means wait longer or read [Troubleshooting](#).

Create the database:

```
gcloud sql databases create $SQL_DB --instance=$SQL_INSTANCE --project=$PROJECT_ID
```

Wait: ~20 seconds. **Success:** `Creating Cloud SQL database...done.`

Read the password out of `.env` and create the user. The password is never typed, echoed, or logged:

```
$PGPASS = ((Get-Content C:\dev\pickready\.env | Where-Object { $_ -match  
"^POSTGRES_PASSWORD=" }) -split "=", 2)[1].Trim().Trim('\"').Trim('\"')
```

```
if ([string]::IsNullOrEmpty($PGPASS)) { Write-Error "POSTGRES_PASSWORD is empty in .env" } else { Write-Host "POSTGRES_PASSWORD loaded, $($PGPASS.Length) chars" }
```

Success: POSTGRES_PASSWORD loaded, 32 chars.

```
gcloud sql users create $SQL_USER --instance=$SQL_INSTANCE --password=$PGPASS --project=$PROJECT_ID
```

Wait: ~15 seconds. **Success:** Creating Cloud SQL user...done.

Capture the connection name, which is what the socket path is built from:

```
$SQL_CONN = (gcloud sql instances describe $SQL_INSTANCE --format="value(connectionName)" --project=$PROJECT_ID).Trim()
```

```
Write-Host "SQL_CONN = $SQL_CONN"
```

Success: SQL_CONN = pick-ready-503913:asia-south1:pickready-postgres.

Build the DSN. `EscapeDataString` is a no-op for the current 32-character password (verified URL-safe), but it is what stops a future rotated password containing @, /, : or # from silently producing an unparseable DSN:

```
$DATABASE_URL = "postgresql+asyncpg://$SQL_USER" + ":" +  
[uri]::EscapeDataString($PGPASS) + "@/$SQL_DB" + "?host=/cloudsql/$SQL_CONN"
```

```
Write-Host $DATABASE_URL.Replace($PGPASS, "*****")
```

Success: postgresql+asyncpg://pickready:*****@/pickready?host=/cloudsql/pick-ready-503913:asia-south1:pickready-postgres

`pgvector` and `pg_trgm` are created by `alembic 0001_initial`, which the migration job runs in Phase 8. There is no manual extension step.

Phase 3: Create Memorystore, Redis 7 (5 minutes)

```
$REDIS_INSTANCE = "pickready-redis"
```



```
gcloud redis instances create $REDIS_INSTANCE --size=1 --region=$REGION --redis-version=redis_7_0 --network=default --project=$PROJECT_ID
```

Wait: 4 to 6 minutes.

Verify it is **READY**:

```
gcloud redis instances describe $REDIS_INSTANCE --region=$REGION --format="value(state)" --project=$PROJECT_ID
```

Success: **READY**.

```
$REDIS_HOST = (gcloud redis instances describe $REDIS_INSTANCE --region=$REGION --format="value(host)" --project=$PROJECT_ID).Trim()
```

```
$REDIS_PORT = (gcloud redis instances describe $REDIS_INSTANCE --region=$REGION --format="value(port)" --project=$PROJECT_ID).Trim()
```

```
$REDIS_URL = "redis://" + $REDIS_HOST + ":" + $REDIS_PORT + "/"
```

```
Write-Host "REDIS_URL = $REDIS_URL"
```

Success: something like `redis://10.x.x.x:6379/`. A **private** 10.x address is correct: this is why the backend and both worker pools need Direct VPC egress. There is no password, so this is a plain env var, not a secret.

Phase 4: Set up Artifact Registry (2 minutes)

```
$REPO_NAME = "pickready"
```

```
gcloud artifacts repositories create $REPO_NAME --repository-format=docker --location=$REGION --description="PickReady images" --project=$PROJECT_ID
```

Wait: ~20 seconds. **Success:** Created repository [pickready].

Give the local Docker daemon a credential helper for this registry:

```
gcloud auth configure-docker "$REGION-docker.pkg.dev" --quiet
```

Success: Docker configuration file updated.

Verify:

```
gcloud artifacts repositories describe $REPO_NAME --location=$REGION --
format="value(name,format)" --project=$PROJECT_ID
```

Success: the repository path and DOCKER.

Phase 5: Push Secrets to Secret Manager (10 minutes)

25 secrets are pushed. Two rules make this correct and both matter:

- **Matched surrounding quotes are stripped.** `FIREBASE_SERVICE_ACCOUNT_JSON` is single-quoted in `.env` and `json.loads` will not accept the quotes.
- **The file is written UTF-8 with no BOM and no trailing newline.** `Set-Content -Encoding utf8` on PowerShell 5.1 writes a **BOM**, which lands inside the secret value and corrupts every consumer. `WriteAllText` with an explicit `UTF8Encoding $false` is what avoids it.

Paste this whole block at once. It reads `.env`, pushes each populated key, and prints one line per secret without ever printing a value.

```
$EnvPath = "C:\dev\pickready\.env"
$SecretKeys =
@("JWT_SECRET", "LLM_KEY_ENCRYPTION_SECRET", "FIREBASE_SERVICE_ACCOUNT_JSON", "SMTP_H
OST", "SMTP_PORT", "SMTP_USER", "SMTP_PASSWORD", "SMTP_FROM_EMAIL", "SMTP_FROM_NAME", "R
AZORPAY_KEY_ID", "RAZORPAY_KEY_SECRET", "RAZORPAY_WEBHOOK_SECRET", "CLOUDINARY_URL", "
TAVILY_API_KEY", "MSG91_API_KEY", "MSG91_SENDER_ID", "BGE_M3_ENDPOINT", "GROQ_API_KEY_
1", "GROQ_API_KEY_2", "GROQ_API_KEY_3", "GROQ_API_KEY_4", "GROQ_API_KEY_5", "GROQ_API_K
EY_6", "GROQ_API_KEY_7", "GEMINI_API_KEY_1", "GEMINI_API_KEY_2", "GEMINI_API_KEY_3", "G
EMINI_API_KEY_4", "GEMINI_API_KEY_5", "GEMINI_API_KEY_6", "GEMINI_API_KEY_7", "OPENROU
TER_API_KEY_1", "OPENROUTER_API_KEY_2", "OPENROUTER_API_KEY_3", "OPENROUTER_API_KEY_4
", "OPENROUTER_API_KEY_5", "OPENROUTER_API_KEY_6", "OPENROUTER_API_KEY_7")
$envMap = @{}
foreach ($line in (Get-Content $EnvPath)) { $t = $line.Trim(); if ($t -eq "" -or
$t.StartsWith("#") -or (-not $t.Contains("="))) { continue }; $i =
$t.IndexOf("="); $k = $t.Substring(0, $i).Trim(); $v = $t.Substring($i +
1).Trim(); if ($v.Length -ge 2 -and (($v.StartsWith('') -and $v.EndsWith('')) -
or ($v.StartsWith('') -and $v.EndsWith('')))) { $v = $v.Substring(1, $v.Length -
2) }; $envMap[$k] = $v }
$utf8NoBom = New-Object System.Text.UTF8Encoding $false
$pushed = 0; $skipped = 0
foreach ($k in $SecretKeys) { $v = $envMap[$k]; if
```

```
([string]::IsNullOrEmpty($v)) { Write-Host "  skip    $k (blank in .env)";
$skipped++; continue }; $tmp = [System.IO.Path]::GetTempFileName();
[System.IO.File]::WriteAllText($tmp, $v, $utf8NoBom); gcloud secrets describe $k -
--project=$PROJECT_ID --format="value(name)" 1>$null 2>$null; if ($LASTEXITCODE -eq
0) { gcloud secrets versions add $k --data-file=$tmp --project=$PROJECT_ID
1>$null; Write-Host "  update $k" } else { gcloud secrets create $k --data-
file=$tmp --replication-policy=automatic --project=$PROJECT_ID 1>$null; Write-Host
"  create $k" }; Remove-Item $tmp -Force; $pushed++ }
Write-Host ""
Write-Host "pushed=$pushed skipped=$skipped"
```

Wait: ~2 seconds per secret, so about 60 seconds. **Success (current .env):** pushed=24 skipped=14.

pushed=24 reflects RAZORPAY_WEBHOOK_SECRET still being blank. See **BLOCKER 1**. After you add it in Phase 15, re-run this block and it becomes pushed=25 skipped=13.

Push DATABASE_URL as a secret

infra/gcp/deploy.sh passes the DSN via --set-env-vars, which puts the database password in plaintext in the service description, readable by anyone with run.viewer. This runbook stores it as a secret instead.

```
$tmp = [System.IO.Path]::GetTempFileName()
```

```
[System.IO.File]::WriteAllText($tmp, $DATABASE_URL, (New-Object
System.Text.UTF8Encoding $false))
```

```
gcloud secrets describe DATABASE_URL --project=$PROJECT_ID --format="value(name)"
1>$null 2>$null ; if ($LASTEXITCODE -eq 0) { gcloud secrets versions add
DATABASE_URL --data-file=$tmp --project=$PROJECT_ID } else { gcloud secrets create
DATABASE_URL --data-file=$tmp --replication-policy=automatic --project=$PROJECT_ID
}
```

```
Remove-Item $tmp -Force
```

Verify the JSON secret survived intact. This is the single most important check in this phase, because a broken value here means nobody can log in:

```
gcloud secrets versions access latest --secret=Firebase_Service_Account_JSON --
project=$PROJECT_ID | ConvertFrom-Json | Select-Object type, project_id
```

Success:

```

type          project_id
----          -
service_account pick-ready

```

A `ConvertFrom-Json` error means the quotes were not stripped. Re-run the block.

Count what landed:

```

gcloud secrets list --project=$PROJECT_ID --format="value(name)" | Measure-Object
-Line

```

Success: Lines: 25 (24 from `.env` plus `DATABASE_URL`).

Phase 6: Build and push Docker images (10 minutes)

```
$REGISTRY = "$REGION-docker.pkg.dev/$PROJECT_ID/$REPO_NAME"
```

```
$TAG = (git -C C:\dev\pickready rev-parse --short HEAD).Trim()
```

```
$IMAGE_BACKEND = "$REGISTRY/backend"
```

```
$IMAGE_FRONTEND = "$REGISTRY/frontend"
```

```
Write-Host "TAG=$TAG"; Write-Host "backend=$IMAGE_BACKEND"; Write-Host
"frontend=$IMAGE_FRONTEND"
```

Tagging by git SHA is what makes `rollback` a one-line redeploy rather than a rebuild.

Build the backend. `--platform linux/amd64` is required: Cloud Run does not run arm64 images, and an Apple-silicon or arm Windows host would otherwise produce one that fails at startup with `exec format error`.

```

docker build --platform linux/amd64 -t "${IMAGE_BACKEND}:${TAG}" -t
"${IMAGE_BACKEND}:latest" C:\dev\pickready\backend

```

Wait: 4 to 7 minutes on a cold cache (it compiles wheels), under 60 seconds warm. **Success:** the last line is naming to `...backend:latest` done.

Load the Firebase build args, then build the frontend:

```
$fe = @{} ; foreach ($line in (Get-Content C:\dev\pickready\frontend\.env.local))
{ $t = $line.Trim(); if ($t -eq "" -or $t.StartsWith("#") -or (-not
$t.Contains("="))) { continue }; $i = $t.IndexOf("=");
$fe[$t.Substring(0,$i).Trim()] = $t.Substring($i+1).Trim().Trim(' ').Trim(' ') }
```

```
Write-Host "firebase project = $($fe['NEXT_PUBLIC_FIREBASE_PROJECT_ID'])"
```

Success: `firebase project = pick-ready`. Blank means `.env.local` is not being parsed and the next command will fail the build on purpose.

```
docker build --platform linux/amd64 --build-arg
NEXT_PUBLIC_FIREBASE_API_KEY=$( $fe['NEXT_PUBLIC_FIREBASE_API_KEY'] ) --build-arg
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=$( $fe['NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN'] ) --
build-arg
NEXT_PUBLIC_FIREBASE_PROJECT_ID=$( $fe['NEXT_PUBLIC_FIREBASE_PROJECT_ID'] ) --build-
arg
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=$( $fe['NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET'] )
--build-arg
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=$( $fe['NEXT_PUBLIC_FIREBASE_MESSAGING_SEN
DER_ID'] ) --build-arg
NEXT_PUBLIC_FIREBASE_APP_ID=$( $fe['NEXT_PUBLIC_FIREBASE_APP_ID'] ) -t
"${IMAGE_FRONTEND}:${TAG}" -t "${IMAGE_FRONTEND}:latest" C:\dev\pickready\frontend
```

Wait: 3 to 6 minutes. **Success:** naming to `...frontend:latest` done. **If it fails with ERROR:** `Firebase web config missing` that is `frontend/Dockerfile:50` working as designed: it refuses to ship an image whose login is dead rather than letting you discover it in production.

Push all four tags:

```
docker push "${IMAGE_BACKEND}:${TAG}" ; docker push "${IMAGE_BACKEND}:latest" ;
docker push "${IMAGE_FRONTEND}:${TAG}" ; docker push "${IMAGE_FRONTEND}:latest"
```

Wait: 3 to 8 minutes, network-bound.

Verify both are in the registry:

```
gcloud artifacts docker images list $REGISTRY --include-tags --  
format="table(package,tags)" --project=$PROJECT_ID
```

Success: two rows, one `.../backend` and one `.../frontend`, each tagged with your `$TAG` and `latest`.

Phase 7: Create the runtime service account (2 minutes)

This is an **identity**, not a key file. Nothing is downloaded. It exists so the running containers hold only the two permissions they need, while deployment rights stay with your user account.

```
$RUNTIME_SA = "pickready-runtime@$PROJECT_ID.iam.gserviceaccount.com"
```

```
gcloud iam service-accounts create pickready-runtime --display-name="PickReady  
Cloud Run runtime" --project=$PROJECT_ID
```

Wait: ~10 seconds. **Success:** Created service account [pickready-runtime].

```
gcloud projects add-iam-policy-binding $PROJECT_ID --  
member="serviceAccount:$RUNTIME_SA" --role="roles/secretmanager.secretAccessor" --  
condition=None
```

```
gcloud projects add-iam-policy-binding $PROJECT_ID --  
member="serviceAccount:$RUNTIME_SA" --role="roles/cloudsql.client" --  
condition=None
```

Wait: ~10 seconds each. **Success:** each prints the updated IAM policy.

Your own account must be allowed to deploy *as* this identity:

```
gcloud iam service-accounts add-iam-policy-binding $RUNTIME_SA --  
member="user:manjuchro@gmail.com" --role="roles/iam.serviceAccountUser" --  
project=$PROJECT_ID
```

Success: prints the updated binding. Skipping this produces `PERMISSION_DENIED: ...
iam.serviceaccounts.actAs` at the first deploy.

Verify:

```
gcloud projects get-iam-policy $PROJECT_ID --flatten="bindings[].members" --
filter="bindings.members:pickready-runtime" --format="value(bindings.role)"
```

Success: exactly two lines, `roles/cloudsql.client` and `roles/secretmanager.secretAccessor`.

Phase 8: Run database migrations (5 minutes)

Migrations run as a **Job**, never on API startup: several instances boot at once during a rollout and would race through the same migration. Alembic takes a lock, so the losers crash-loop.

Build the secrets flag once. It maps every secret to the env var of the same name, and is reused by the job, the backend and both pools:

```
$SECRET_NAMES = (gcloud secrets list --project=$PROJECT_ID --format="value(name)")
```

```
$SECRET_FLAG = (($SECRET_NAMES | ForEach-Object { "$_=$_" + ":latest" }) -join
",")
```

```
Write-Host "$($SECRET_NAMES.Count) secrets will be mounted"
```

Success: 25 secrets will be mounted.

```
gcloud run jobs create pickready-migrate --image="${IMAGE_BACKEND}:${TAG}" --
region=$REGION --service-account=$RUNTIME_SA --set-cloudsql-instances=$SQL_CONN --
set-env-vars="ENVIRONMENT=production,REDIS_URL=$REDIS_URL" --set-
secrets=$SECRET_FLAG --args=migrate --max-retries=1 --task-timeout=900s --
project=$PROJECT_ID
```

Wait: ~30 seconds. **Success:** Job [pickready-migrate] has been created.

Execute it and block until it finishes:

```
gcloud run jobs execute pickready-migrate --region=$REGION --wait --
project=$PROJECT_ID
```

Wait: 60 to 180 seconds. This runs all 30 migrations including `0030_ppi_framework`. **Success:** Execution [pickready-migrate-xxxxx] has successfully completed.

If it fails, read the logs before changing anything:

```
gcloud run jobs executions list --job=pickready-migrate --region=$REGION --
format="value(name)" --project=$PROJECT_ID | Select-Object -First 1
```

```
gcloud logging read "resource.type=cloud_run_job AND
resource.labels.job_name=pickready-migrate" --limit=50 --
format="value(textPayload)" --project=$PROJECT_ID
```

Phase 9: Deploy the backend service (5 minutes)

Public on purpose: Razorpay posts webhooks straight to it, and the application authenticates every other route itself.

```
gcloud run deploy pickready-backend --image="${IMAGE_BACKEND}:${TAG}" --
region=$REGION --platform=managed --service-account=$RUNTIME_SA --allow-
unauthenticated --memory=2Gi --cpu=2 --timeout=300 --min-instances=0 --max-
instances=10 --network=default --subnet=default --vpc-egress=private-ranges-only -
-add-cloudsql-instances=$SQL_CONN --set-env-
vars="ENVIRONMENT=production,REDIS_URL=$REDIS_URL" --set-secrets=$SECRET_FLAG --
args=api --project=$PROJECT_ID
```

Wait: 90 to 180 seconds. **Success:** Service [pickready-backend] revision [pickready-backend-00001-xxx] has been deployed and is serving 100 percent of traffic.

```
$BACKEND_URL = (gcloud run services describe pickready-backend --region=$REGION --
format="value(status.url)" --project=$PROJECT_ID).Trim()
```

```
Write-Host "BACKEND_URL = $BACKEND_URL"
```

Test the health endpoint. **The path is /health.** /api/v1/health does not exist and returns 404:

```
(Invoke-WebRequest -Uri "$BACKEND_URL/health" -UseBasicParsing).Content
```

Success: {"status":"ok"}.

Confirm the whole route table loaded, which proves the database and secrets are reachable, not just that the process started:


```
((Invoke-WebRequest -Uri "$BACKEND_URL/openapi.json" -UseBasicParsing).Content |
ConvertFrom-Json).paths.PSObject.Properties.Name.Count
```

Success: 189.

Phase 10: Deploy the frontend service (5 minutes)

`BACKEND_INTERNAL_URL` is what the same-origin proxy forwards to. It is read per request by `frontend/app/api/[...path]/route.ts`, never inlined into the bundle, and the browser never sees it. `NEXT_PUBLIC_API_URL` must stay **unset**: setting it opts into split-origin mode and the auth cookies are dropped.

The frontend gets **no secrets and no Cloud SQL**. It holds nothing sensitive.

```
gcloud run deploy pickready-frontend --image="${IMAGE_FRONTEND}:${TAG}" --
region=$REGION --platform=managed --service-account=$RUNTIME_SA --allow-
unauthenticated --memory=1Gi --cpu=1 --min-instances=0 --max-instances=10 --set-
env-vars="BACKEND_INTERNAL_URL=$BACKEND_URL,NODE_ENV=production" --
project=$PROJECT_ID
```

Wait: 60 to 120 seconds.

```
$FRONTEND_URL = (gcloud run services describe pickready-frontend --region=$REGION
--format="value(status.url)" --project=$PROJECT_ID).Trim()
```

```
Write-Host "FRONTEND_URL = $FRONTEND_URL"
```

```
(Invoke-WebRequest -Uri $FRONTEND_URL -UseBasicParsing).StatusCode
```

Success: 200.

Prove the proxy reaches the backend, which is the thing most likely to be misconfigured:

```
(Invoke-WebRequest -Uri "$FRONTEND_URL/api/v1/billing/config" -
UseBasicParsing).Content
```

Success: JSON containing `"razorpay_key_id"` and `"plans"`. A 502 or an HTML page means `BACKEND_INTERNAL_URL` did not take.

Phase 11: Publish the frontend URL to the backend (1 minute)

The backend needs the public origin for two things: the links it puts in the six lifecycle emails, and the CORS allowlist. Neither is knowable until the frontend exists, which is why this is a second pass rather than part of Phase 9.

```
gcloud run services update pickready-backend --region=$REGION --update-env-  
vars="FRONTEND_URL=$FRONTEND_URL" --project=$PROJECT_ID
```

Wait: 45 to 90 seconds (this creates a new revision).

Verify:

```
gcloud run services describe pickready-backend --region=$REGION --  
format="value(spec.template.spec.containers[0].env)" --project=$PROJECT_ID |  
Select-String "FRONTEND_URL"
```

Success: a line containing your frontend URL.

Phase 12: Deploy the Celery worker pool (3 minutes)

If you skipped [Phase 0](#), this is where it fails. Set it now if `$env:CLOUDSDK_PYTHON_SITEPACKAGES` is unset:

```
$env:CLOUDSDK_PYTHON_SITEPACKAGES = "1"
```

Worker pools, **not** services: a Celery worker serves no HTTP, so a service would never pass its startup probe and every revision would roll back.

```
gcloud run worker-pools deploy pickready-worker --image="${IMAGE_BACKEND}:${TAG}"  
--region=$REGION --service-account=$RUNTIME_SA --network=default --subnet=default  
--add-cloudsql-instances=$SQL_CONN --set-env-  
vars="ENVIRONMENT=production,REDIS_URL=$REDIS_URL,FRONTEND_URL=$FRONTEND_URL" --  
set-secrets=$SECRET_FLAG --memory=2Gi --cpu=2 --instances=1 --args=worker --  
project=$PROJECT_ID
```

Wait: 60 to 120 seconds. **Success:** Worker pool [pickready-worker] has been deployed.

Verify:

```
gcloud run worker-pools list --region=$REGION --project=$PROJECT_ID
```

Success: `pickready-worker` listed.

Confirm the worker actually connected to Redis and registered its tasks:

```
gcloud logging read "resource.type=cloud_run_worker_pool AND
resource.labels.worker_pool_name=pickready-worker" --limit=30 --
format="value(textPayload)" --project=$PROJECT_ID | Select-String
"celery@|Connected to redis"
```

Success: `Connected to redis://10.x.x.x:6379/0` and `celery@... ready`. Neither appearing means VPC egress is not reaching Memorystore.

Phase 13: Deploy the Celery beat pool (3 minutes)

--instances=1, always. Two beat schedulers means every periodic task fires twice: the dashboard refresh, the assessment-setup reminder, and the credit reconciliation sweep would each run double. The reconciliation sweep is idempotent, but the reminder email is not.

```
gcloud run worker-pools deploy pickready-beat --image="${IMAGE_BACKEND}:${TAG}" --
region=$REGION --service-account=$RUNTIME_SA --network=default --subnet=default --
add-cloudsql-instances=$SQL_CONN --set-env-
vars="ENVIRONMENT=production,REDIS_URL=$REDIS_URL,FRONTEND_URL=$FRONTEND_URL" --
set-secrets=$SECRET_FLAG --memory=512Mi --cpu=1 --instances=1 --args=beat --
project=$PROJECT_ID
```

Wait: 45 to 90 seconds.

Verify it is scheduling, and that the instance count really is 1:

```
gcloud run worker-pools describe pickready-beat --region=$REGION --
format="value(spec.template.spec.containers[0].resources.limits,status.observedGen
eration)" --project=$PROJECT_ID
```

```
gcloud logging read "resource.type=cloud_run_worker_pool AND
resource.labels.worker_pool_name=pickready-beat" --limit=20 --
format="value(textPayload)" --project=$PROJECT_ID | Select-String "beat:
Starting|Scheduler"
```

Success: `beat: Starting...` Beat writes its schedule to `/tmp/celerybeat-schedule` because the image's source tree is read-only to the runtime user.

Phase 14: Verify all services (5 minutes)

```
gcloud run services list --region=$REGION --  
format="table(SERVICE, LAST_DEPLOYED_BY, URL)" --project=$PROJECT_ID
```

Success: `pickready-backend` and `pickready-frontend`.

```
gcloud run worker-pools list --region=$REGION --project=$PROJECT_ID
```

Success: `pickready-worker` and `pickready-beat`.

```
gcloud run jobs list --region=$REGION --project=$PROJECT_ID
```

Success: `pickready-migrate`.

```
(Invoke-WebRequest -Uri "$BACKEND_URL/health" -UseBasicParsing).Content
```

Success: `{"status":"ok"}`.

```
(Invoke-WebRequest -Uri $FRONTEND_URL -UseBasicParsing).StatusCode
```

Success: `200`.

Check for errors across everything in the last 15 minutes:

```
gcloud logging read "severity>=ERROR AND (resource.type=cloud_run_revision OR  
resource.type=cloud_run_worker_pool)" --freshness=15m --limit=50 --  
format="value(resource.labels.service_name,textPayload)" --project=$PROJECT_ID
```

Success: no output. Anything here is real; take it to [Troubleshooting](#).

Phase 15: Post-deployment manual setup (10 minutes)

Neither of these can be done through `gcloud`, and the product is not usable until both are.

1. Firebase authorised domain

Why: Firebase refuses to complete a sign-in initiated from an origin it does not know. Until this is done, every Google sign-in fails with `auth/authorized-domain` and email/password sign-in fails too.

How: Firebase Console > project `pick-ready` > Authentication > Settings > Authorised domains > Add domain. Add the frontend **hostname only**, with no scheme and no trailing slash:

```
$FRONTEND_URL.Replace("https://", "")
```

Paste that exact value.

2. Razorpay webhook

Why: Razorpay charges the customer's card and then tells PickReady by posting to this URL. Without it, subscriptions are charged and no credits are ever granted. This is also the only reason the backend is public.

How: Razorpay Dashboard > Settings > Webhooks > Add New Webhook.

- **URL:**

```
Write-Host "$BACKEND_URL/api/v1/billing/webhook/razorpay"
```

Note the trailing `/razorpay`. `.../billing/webhook` does not exist and returns 404.

(`infra/gcp/deploy.sh:365` prints the shorter, wrong path.)

- **Active events:** `subscription.charged`, `subscription.halted`, `subscription.cancelled`, `payment.failed`.
- **Secret:** generate one, then put it in `.env` as `RAZORPAY_WEBHOOK_SECRET` and re-run the Phase 5 block, then redeploy the backend so it picks up the new secret version:

```
gcloud run services update pickready-backend --region=$REGION --  
project=$PROJECT_ID
```

This closes BLOCKER 1. Until it is done, every webhook is rejected with 400.

3. Smoke-test the real flows

Open `$FRONTEND_URL` in a browser and confirm:

- The landing page renders and the PPI Assessment Report section is present.
- Sign-in with Google completes and **stays** signed in after a refresh. An immediate logout means the auth cookie was dropped; see Troubleshooting.
- `$FRONTEND_URL/docs` renders.

Troubleshooting Guide

Backend will not start

```
gcloud run services logs read pickready-backend --region=$REGION --limit=100 --
project=$PROJECT_ID
```

It is `gcloud run services logs read`, not `gcloud run logs read`.

Log line	Cause	Fix
Revision is not ready ... failed to start and listen on \$PORT	Container is not binding <code>0.0.0.0:\$PORT</code>	The entrypoint handles this; if you overrode <code>--args</code> , put it back to <code>api</code>
Permission denied on secret	Runtime SA lacks <code>secretAccessor</code>	Re-run the Phase 7 binding
FIREBASE_SERVICE_ACCOUNT_JSON is not configured	Secret missing or empty	Re-run Phase 5, then verify with the <code>ConvertFrom-Json</code> check
Firebase Admin could not be initialized	The JSON has surrounding quotes	BLOCKER 2. Re-run the Phase 5 block, which strips them
connection refused / asyncpg timeout	Wrong DSN shape	It must be <code>postgresql+asyncpg://...@/db?host=/cloudsql/CONN</code> , not <code>host:port</code>

Database migration failed

```
gcloud logging read "resource.type=cloud_run_job AND
resource.labels.job_name=pickready-migrate AND severity>=ERROR" --limit=50 --
format="value(textPayload)" --project=$PROJECT_ID
```

Symptom	Cause	Fix
<code>CREATE EXTENSION vector</code> fails	pgvector unavailable on that version	Instance must be Postgres 15 or newer; this runbook creates 16
password authentication failed	POSTGRES_PASSWORD in <code>.env</code> differs from the SQL user	<code>gcloud sql users set-password pickready -</code> <code>-instance=\$SQL_INSTANCE --</code> <code>password=\$PGPASS</code> , then rebuild <code>DATABASE_URL</code> and re-push the secret
Hangs then times out at 900s	No <code>--set-cloudsql-instances</code> on the job	Recreate the job with that flag
Target database is not up to date	A prior partial run	<code>gcloud run jobs execute pickready-migrate</code> <code>--region=\$REGION --wait</code> again; alembic is idempotent

Frontend loads but login fails

Symptom	Cause	Fix
<code>auth/Unauthorized-domain</code>	Frontend host not in Firebase	Phase 15 step 1
<code>auth/invalid-api-key</code>	Build args missing at image build	Rebuild with Phase 6; the Dockerfile fails fast on this
Login succeeds, instantly logged out	Auth cookie dropped, split origin	Confirm <code>NEXT_PUBLIC_API_URL</code> is unset on the frontend service
API calls return HTML instead of JSON	<code>BACKEND_INTERNAL_URL</code> unset, proxy 502s	Re-run Phase 10, then Phase 11

Check the frontend's env quickly:

```
gcloud run services describe pickready-frontend --region=$REGION --
format="value(spec.template.spec.containers[0].env)" --project=$PROJECT_ID
```

Workers not processing tasks

Symptom	Cause	Fix
No <code>Connected to redis</code> in logs	No VPC egress	Redeploy the pool with <code>--network=default</code> <code>--subnet=default</code>
No module named <code>'grpc'</code> at deploy	gcloud site-packages	Phase 0
Deployed but idle	Deployed as a service, not a worker pool	Delete it and redeploy with <code>gcloud run worker-pools deploy</code>
Tasks run twice	More than one beat instance	<code>--instances=1</code> on <code>pickready-beat</code> , always

Secrets not found

```
gcloud secrets list --project=$PROJECT_ID --format="value(name)"
```

```
gcloud projects get-iam-policy $PROJECT_ID --flatten="bindings[].members" --
filter="bindings.members:pickready-runtime AND bindings.role:secretmanager" --
format="value(bindings.role)"
```

Success: `roles/secretmanager.secretAccessor`. If absent, re-run Phase 7. A secret added *after* a deploy needs a new revision to be picked up:

```
gcloud run services update pickready-backend --region=$REGION --  
project=$PROJECT_ID
```

Rollback Procedure

Images are tagged with the git SHA, so rolling back is a redeploy of an earlier tag, never a rebuild.

Option A, instant traffic shift (preferred). The previous revision is still there. List revisions newest first:

```
gcloud run revisions list --service=pickready-backend --region=$REGION --  
format="table(REVISION,ACTIVE,CREATED)" --project=$PROJECT_ID
```

Send all traffic to the previous one:

```
gcloud run services update-traffic pickready-backend --to-  
revisions=PASTE_REVISION_NAME=100 --region=$REGION --project=$PROJECT_ID
```

Wait: ~10 seconds. This is the fastest rollback available and needs no image work at all.

Option B, canary then cut over. Deploy the new revision without traffic, verify it, then shift:

```
gcloud run deploy pickready-backend --image="${IMAGE_BACKEND}:${TAG}" --  
region=$REGION --no-traffic --tag=candidate --project=$PROJECT_ID
```

```
gcloud run services describe pickready-backend --region=$REGION --  
format="value(status.traffic)" --project=$PROJECT_ID
```

The candidate gets its own URL (`candidate---pickready-backend-...`). Test it, then split 10 percent to it:

```
gcloud run services update-traffic pickready-backend --to-tags=candidate=10 --  
region=$REGION --project=$PROJECT_ID
```

Promote to 100, or abandon:


```
gcloud run services update-traffic pickready-backend --to-latest --region=$REGION
--project=$PROJECT_ID
```

Option C, redeploy an older image.

```
gcloud run deploy pickready-backend --image="${IMAGE_BACKEND}:OLD_SHA" --
region=$REGION --project=$PROJECT_ID
```

Database migrations do not roll back with the image. Alembic has a `downgrade` path, but running it against production data is a data-loss risk. The additive migration convention in this repo means an older image against a newer schema generally still works. Prefer rolling forward.

Worker pools roll back the same way:

```
gcloud run worker-pools deploy pickready-worker --image="${IMAGE_BACKEND}:OLD_SHA"
--region=$REGION --project=$PROJECT_ID
```

Monitoring and Alerts

Live tail

```
gcloud beta run services logs tail pickready-backend --region=$REGION --
project=$PROJECT_ID
```

Requires the `beta` component. Installing it needs an elevated prompt on this machine because the SDK lives under `Program Files (x86)`. Without it, poll:

```
gcloud run services logs read pickready-backend --region=$REGION --limit=50 --
project=$PROJECT_ID
```

Errors across every workload

```
gcloud logging read "severity>=ERROR AND (resource.type=cloud_run_revision OR
resource.type=cloud_run_worker_pool OR resource.type=cloud_run_job)" --
freshness=1h --limit=100 --
format="table(timestamp,resource.labels.service_name,textPayload)" --
project=$PROJECT_ID
```

Application-specific signals worth watching

These are logged by name and are the ones that indicate a degraded product rather than a crash:

```
gcloud logging read "resource.type=cloud_run_revision AND textPayload:
(\"llm_router.provider_degraded\" OR \"scoring_mode\" OR \"delivery.exhausted\")"
--freshness=6h --limit=50 --format="value(timestamp,textPayload)" --
project=$PROJECT_ID
```

- `llm_router.provider_degraded` means an LLM tier is failing over.
- `functional_assessment.scoring_mode` with `deterministic_fallback` means reports are being written without the model.
- `delivery.exhausted` means an email or SMS gave up after its retries.

A basic uptime alert

```
gcloud alpha monitoring channels create --display-name="PickReady ops" --
type=email --channel-labels=email_address=manjuchro@gmail.com --
project=$PROJECT_ID
```

Then create an uptime check against `$BACKEND_URL/health` in Cloud Console > Monitoring > Uptime checks. A check on `/` would follow the frontend's redirects and pass even when the API is down, which is why it points at `/health`.

Summary

Total time	~2 hours, ~25 minutes of it hands-on
Longest step	Cloud SQL creation, 7 to 12 minutes
Workloads	2 services, 2 worker pools, 1 job
Secrets	25 in Secret Manager, 0 in env vars
Service-account keys	none

Print the URLs:

```
Write-Host "Frontend : $FRONTEND_URL" ; Write-Host "Backend : $BACKEND_URL" ;
Write-Host "Webhook : $BACKEND_URL/api/v1/billing/webhook/razorpay"
```

Running cost

Resource	Spec	Approx / month
Cloud SQL <code>db-custom-1-3840</code>	1 vCPU, 3.75 GB	~\$50
Memorystore Basic	1 GB	~\$25
Worker + beat pools	1 instance each, always on	~\$25
Cloud Run services	scale to zero	usage only
Artifact Registry	a few GB	~\$1
Total		~\$100 to \$110

The two worker pools are the one thing that cannot scale to zero: Celery has to be running when a task is queued.

Next steps

1. Close **BLOCKER 1** by setting `RAZORPAY_WEBHOOK_SECRET` (Phase 15 step 2).
2. Seed the first customer tenant and an owner account.
3. Map a custom domain: `gcloud run domain-mappings create --service=pickready-frontend -domain=app.yourdomain.com --region=$REGION`, then add that domain to Firebase authorised domains too.
4. Set `--min-instances=1` on the backend if cold starts are noticeable.

Redeploying after a code change

```
$TAG = (git -C C:\dev\pickready rev-parse --short HEAD).Trim()
```

Then Phase 6 (build and push), Phase 8 (migrate, only if there are new migrations), and Phases 9, 12 and 13 (backend, worker, beat). The frontend only needs rebuilding if frontend code or the Firebase project changed.

Appendix A: Corrections to the existing tooling

Three defects were found in `infra/gcp/deploy.sh` and `docs/DEPLOY_GCP.md` while verifying this runbook against the code. They are listed here so they can be fixed at the source; this runbook already works around all three.

1. **Wrong webhook path.** `deploy.sh:365` and `DEPLOY_GCP.md` section 4 print `${BACKEND_URL}/api/v1/billing/webhook`. The route is `/api/v1/billing/webhook/razorpay` (`api/billing.py:581`). A webhook configured from that instruction 404s on every delivery.
2. **Quoted secrets are pushed verbatim.** `deploy.sh:97` reads values with `sed -n "s/^KEY=//p"`, which does not strip surrounding quotes. `FIREBASE_SERVICE_ACCOUNT_JSON` is single-quoted in `.env`, so the script stores invalid JSON and Firebase Admin cannot initialise in production.
3. **The database password is stored as a plain env var.** `deploy.sh:311` passes the full DSN through `--set-env-vars`, making the password readable to anyone with `run.viewer`. This runbook stores

`DATABASE_URL` in Secret Manager instead.